

Feuille de travaux dirigés n° 3

Moniteurs de synchronisation **— CORRECTION —**

Exercice 1 (La pile bloquante).

Une pile bloquante bornée, dont la capacité est passée en paramètre au constructeur, possède deux opérations :

- `void push(T)` attend qu'il y ait de la place dans la pile puis ajoute un élément en tête de la pile ;
- `T pop()` attend que la pile ne soit pas vide puis supprime et retourne l'élément en tête de la pile.

Implémentez une pile bloquante bornée en Java.

Exercice 2 (Les toilettes unisexes¹).

Les toilettes unisexes peuvent être utilisées par les hommes et par les femmes, sous les contraintes suivantes :

1. il ne doit jamais y avoir en même temps des hommes et des femmes aux toilettes ;
2. il ne doit jamais y avoir plus de trois personnes simultanément aux toilettes.

Un programme modélisant le problème est disponible sous Madoc. Le comportement des personnes qui accèdent aux toilettes est déjà modélisé. Les toilettes doivent implémenter l'interface `Bathroom`, qui contient une méthode `void enter(boolean isMale)` et une méthode `void leave(boolean isMale)`.

2a. Proposez une solution deadlock-free.

2b. Modifiez votre solution pour la rendre starvation-free.

Indice : des humains bien élevés organiseraient probablement une file d'attente.

Exercice 3 (Implémentation d'un thread pool).

On rappelle les interfaces (partielles) des types permettant de gérer les thread pools en Java.

```
interface Callable<T> {  
    T call();  
}  
interface Future<T> {  
    boolean isDone();  
    T get(); // Met le thread en attente jusqu'à la fin de la tâche  
}  
interface ThreadPool { Future<T> submit(Callable<T> task); }
```

3a. Proposez une implémentation de `Future` et de `ThreadPool`, permettant de gérer un thread pool de taille fixe (le nombre de threads sera passé en paramètre au constructeur).

On souhaite modifier la signature de `call()` et de `get()` en ajoutant `throws Exception`, pour autoriser les tâches à lever des exceptions. Si une tâche lève une exception, elle doit pouvoir être gérée lors de la récupération du résultat.

3b. Modifiez votre code pour prendre en compte les exceptions.

1. Inspiration : A. B. Downey. *The Little Book of Semaphores*. Green Tea Press (2008)

Exercice 4 (Barrière de synchronisation).

Le but de cet exercice est d'implémenter une barrière de synchronisation permettant de s'assurer qu'un certain nombre de threads n aient tous terminé une première phase avant de passer à la seconde. Plus précisément, la barrière contient un compteur initialisé avec le paramètre n de son constructeur, et expose une méthode `void await()`. Quand un thread appelle la méthode `await`, le compteur est décrémenté et le thread appelant est mis en attente tant que le compteur est strictement positif. Autrement dit, les n premiers threads appelant la méthode `await` doivent attendre qu'au moins n threads l'aient appelée. Quand n threads ou plus ont appelé la méthode `await`, tous les threads sont libérés et peuvent continuer leur exécution.

4a. Donnez une implémentation en Java d'une barrière de synchronisation, sous la forme d'un moniteur.

Exercice 5 (Bande de sauvages²).

Une tribu de sauvages mange son dîner dans un pot commun qui peut contenir M parts de potage. Quand un sauvage veut manger, il se sert au pot, sauf s'il est vide. Quand le pot est vide, les sauvages réveillent le cuisinier et attendent qu'il remplisse le pot.

Programmer le comportement des sauvages et du cuisinier. Les contraintes sont les suivantes :

- Les sauvages ne peuvent pas se servir dans un pot vide ;
- Le cuisinier ne doit remplir le pot que quand il est vide ;
- Discutez les propriétés de vivacité obtenues.

Exercice 6 (Le mot-clé `select` en go).

En Go, un canal (*channel*) est une construction du langage, proche d'une `BlockingQueue` en Java, qui permet de transférer des informations entre goroutines et qui peut bloquer jusqu'à ce qu'un élément soit disponible (lecture) ou qu'une place se libère (écriture). L'instruction `select` permet d'attendre plusieurs opérations sur des canaux et de débloquer dès que l'une d'elles est prête. Par exemple, le programme suivant affiche le premier message reçu, soit depuis `channel1`, soit depuis `channel2`.

```
select {  
  case msg1 := <-channel1: fmt.Println(msg1)  
  case msg2 := <-channel2: fmt.Println(msg2)  
}
```

6a. Il n'existe pas d'équivalent direct en Java. Concevez une abstraction qui permette d'attendre sur plusieurs files bloquantes (type `BlockingQueue`) et de reprendre dès que l'une d'elles est prête.

2. Inspiration : G. R. Andrews. *Foundations of Multithreaded, Parallel, and Distributed Programming*. Addison-Wesley (2000)